

✓ A/B Тест: Нагадування про поповнення кредитів

Контекст задачі

Ми — аналітики в команді онбордингу. Ціль зони — **збільшити конверсію в першу оплату**.

В додатку користувачі спілкуються за внутрішню валюту — кредити. При реєстрації кожен отримує **20 бонусних кредитів**. Коли кредити закінчуються, з'являється повідомлення "*недостатньо кредитів на рахунок*" — і саме тут багато користувачів іде, не здійснивши оплату.

Тестова гіпотеза: якщо нагадати користувачу про необхідність поповнення *до* того, як кредити закінчаться, він з більшою ймовірністю зробить першу оплату.

Механіка тесту: закріплене повідомлення в чаті з'являється, коли залишається ≤ 5 кредитів. Зникає після першої оплати і більше не відтворюється.

Гіпотези та параметри

- **H₀:** Конверсія в першу оплату однакова в обох групах: $p_{control} = p_{test}$
- **H₁:** Конверсія в тестовій групі вища: $p_{test} > p_{control}$

Параметр	Значення
Рівень значущості α	0.05
Потужність $(1-\beta)$	0.80
Тип тесту	Одностороній z-тест для пропорцій
Спліт	50% / 50%

Чому одностороній тест? Ми запускаємо нагадування з чіткою гіпотезою, що воно підвищить конверсію. Нас не цікавить сценарій "тест погіршив результат" з точки зору прийняття рішення — тому одностороній тест є коректним і дає більшу потужність при тому ж α .

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import scipy.stats as stats
from statsmodels.stats.proportion import proportions_ztest, proportion_effectsize
from statsmodels.stats.power import NormalIndPower

# Завантаження даних
hist = pd.read_csv('ab_test_task_historical_data.csv')
test = pd.read_csv('ab_test_task_data.csv')

# Конвертація дат
for col in ['date_reg', 'date_first_payment', 'date_spent_15_credits']:
    hist[col] = pd.to_datetime(hist[col], utc=True, errors='coerce')

for col in ['date_reg', 'date_first_payment', 'date_spent_15_credits', 'date_reminder']:
    test[col] = pd.to_datetime(test[col], utc=True, errors='coerce')

print('historical_data:', hist.shape)
print('test_data:', test.shape)
display(hist.head(3))
display(test.head(3))
```

```
historical_data: (59758, 4)
test_data:      (125217, 6)
```

	id_user	date_reg	date_first_payment	date_spent_15_credits	
0	1	2023-03-01 00:00:09.068945+00:00	2023-03-01 02:06:31.472018+00:00	2023-03-01 02:00:38+00:00	
1	2	2023-03-01 00:00:25.615471+00:00	NaT	2023-03-01 00:17:38+00:00	
2	3	2023-03-01 00:00:39.084989+00:00	2023-03-01 00:52:37.812217+00:00	2023-03-01 00:18:53+00:00	
	id_user	date_reg	date_first_payment	date_spent_15_credits	date_reminder match
0	1	2023-03-14 08:37:01.855460+00:00	2023-03-14 11:25:23.962741+00:00	2023-03-14 09:15:31+00:00	2023-03-14 09:04:03.823970+00:00 1
1	2	2023-03-14 08:37:20.988023+00:00	NaT	NaT	NaT 0

Що маємо в даних:

- `historical_data` — 59 758 користувачів до запуску тесту. Містить дату реєстрації, першої оплати та витрати 15 кредитів. Використовуємо для розрахунку базової конверсії та тривалості тесту.
- `test_data` — 125 217 користувачів з моменту запуску тесту. Додатково є `date_reminder` (коли побачив нагадування) та `match` (0 = control, 1 = test).

Завдання 1: Скільки днів потрібно для тесту?

Перш ніж запускати тест, потрібно розрахувати **мінімальну тривалість**, щоб результати були статистично надійними. Логіка розрахунку:

- Беремо базову конверсію з historical data — це наш орієнтир
- Визначаємо MDE (Minimum Detectable Effect) — мінімальний приріст, який для нас практично значущий. Обираємо 10% відносний — тобто якщо поточна CR = 5.45%, то ціль = 5.99%. Менший ефект детектувати або нереалістично, або не варто витрат на розробку
- Через power analysis знаходимо потрібну кількість користувачів на групу
- Ділимо на денний приріст нових юзерів при спліті 50/50 → отримуємо кількість днів

```
# Базова конверсія з historical data
base_cr = hist['date_first_payment'].notna().mean()
print(f'Базова конверсія: {base_cr*100:.2f}%')

# Середній денний приріст користувачів
avg_daily = hist.groupby(hist['date_reg'].dt.date)['id_user'].count().mean()
print(f'Середній приріст: {avg_daily:.0f} юзерів/день')

# Power analysis
p_control = base_cr
p_test = base_cr * 1.10
effect_size = proportion_effectsize(p_control, p_test)

n_per_group = NormalIndPower().solve_power(
    effect_size=abs(effect_size),
    alpha=0.05,
    power=0.80,
    alternative='larger'
)
n_per_group = int(np.ceil(n_per_group))
days_needed = int(np.ceil(n_per_group / (avg_daily * 0.5)))

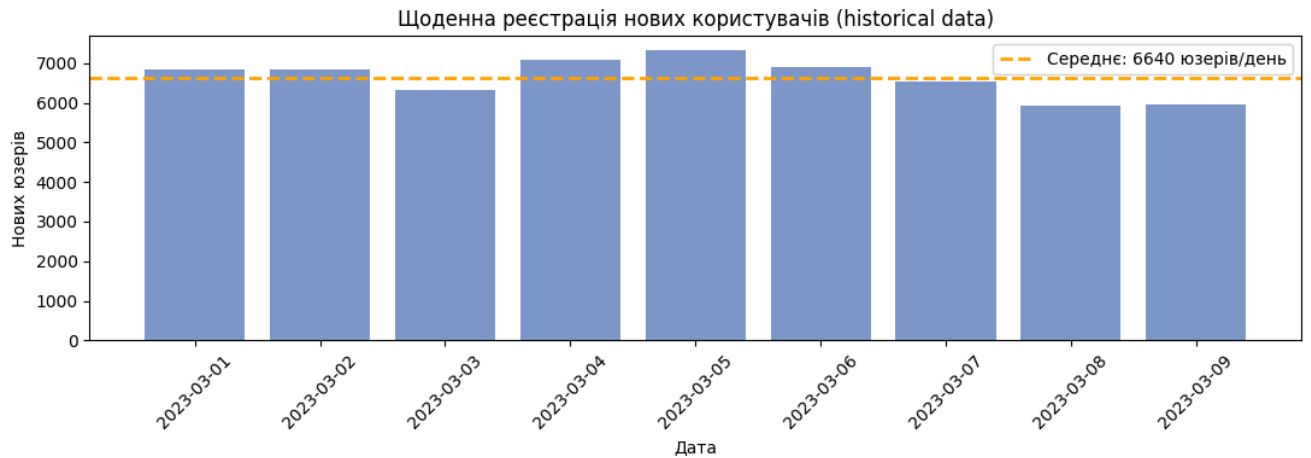
print(f'\nПри MDE=10% відносний:')
print(f'Потрібно на групу: {n_per_group:}, юзерів')
print(f'Потрібно днів: {days_needed}')
```

Базова конверсія: 5.45%
Середній приріст: 6640 юзерів/день

При MDE=10% відносний:
Потрібно на групу: 22,446 юзерів
Потрібно днів: 7

```
# Графік: денний приріст юзерів
daily_reg = hist.groupby(hist['date_reg'].dt.date)['id_user'].count()

plt.figure(figsize=(11, 4))
plt.bar(daily_reg.index, daily_reg.values, color='#4870B6', alpha=0.7)
plt.axhline(avg_daily, color='orange', linestyle='--', linewidth=2,
            label=f'Середнє: {avg_daily:.0f} юзерів/день')
plt.title('Щоденна реєстрація нових користувачів (historical data)')
plt.xlabel('Дата')
plt.ylabel('Нових юзерів')
plt.xticks(rotation=45)
plt.legend()
plt.tight_layout()
plt.show()
```



Показник	Значення
Базова конверсія (historical)	5.45%
Обраний MDE	10% відносний (5.45% → 5.99%)
Потрібно користувачів на групу	22 446
Середній денний приріст	6 640 юзерів/день
Потрібно днів тесту	~7 днів

При рівномірному потоці реєстрацій (6 640/день) та спліті 50/50 кожна група набирає 3 320 юзерів щодня. Для накопичення 22 446 юзерів у кожній групі потрібно **7 днів**.

Важливо: 7 днів — мінімум для детекції ефекту $\geq 10\%$. Якщо реальний ефект менший — тест потрібно буде тримати довше або переглянути MDE.

```
# Фактична тривалість тесту
test_start = test['date_reg'].min()
test_end = test['date_reg'].max()
actual_days = (test_end - test_start).days

print(f'Тест стартував: {test_start.date()}')
print(f'Останній юзер: {test_end.date()}')
print(f'Фактична тривалість: {actual_days} днів')
print(f'Мінімально необхідно: {days_needed} днів')
print()
if actual_days >= days_needed:
    print(f'Тест тривав достатньо — можна довіряти результатам.')
else:
    print(f'Тест тривав менше за необхідний мінімум — результати попередні.')
```

```
Тест стартував: 2023-03-14
Останній юзер: 2023-04-18
Фактична тривалість: 34 днів
Мінімально необхідно: 7 днів
```

```
Тест тривав достатньо — можна довіряти результатам.
```

Фактична тривалість тесту **перевищує розрахований мінімум у 7 днів** — отже вибірка достатня і можна переходити до аналізу результатів.

Завдання 2: Оцінка результатів тесту

Крок 1: SRM Check (Sample Ratio Mismatch)

Перш ніж аналізувати метрики, перевіряємо чи правильно розподілились користувачі по групах. Якщо спліт не 50/50 — це сигнал про технічну помилку, і результатам довіряти не можна.

```
# SRM Check
n_ctrl = (test['match'] == 0).sum()
n_trt = (test['match'] == 1).sum()
expected = (n_ctrl + n_trt) / 2

chi2, p_srm = stats.chisquare([n_ctrl, n_trt], f_exp=[expected, expected])

print(f'Control: {n_ctrl:,} | Test: {n_trt:,} | Очікувалось: {expected:,.0f}')
print(f'p-value SRM: {p_srm:.4f}')
```

```
Control: 62,568 | Test: 62,649 | Очікувалось: 62,608
p-value SRM: 0.8189
```

Control: 62 568 | Test: 62 649 | p-value = 0.8189 >= 0.05

SRM не виявлено. Розподіл практично ідеальний (50.03% / 49.97%) — тест налаштований коректно, продовжуємо аналіз.

Крок 2: Основна метрика — конверсія в першу оплату

Основна метрика тесту — частка користувачів, які здійснили першу оплату (`date_first_payment` не пуста). Порівнюємо між групами за допомогою одностороннього z-тесту.

```
# Конверсія в першу оплату
ctrl = test[test['match'] == 0]
trt = test[test['match'] == 1]

x_ctrl = ctrl['date_first_payment'].notna().sum()
x_trt = trt['date_first_payment'].notna().sum()
cr_ctrl = x_ctrl / n_ctrl
cr_trt = x_trt / n_trt

print(f'Control CR: {x_ctrl}/{n_ctrl} = {cr_ctrl*100:.2f}%')
print(f'Test CR: {x_trt}/{n_trt} = {cr_trt*100:.2f}%')
print(f'Абс. різниця: {(cr_trt-cr_ctrl)*100:+.2f} п.п.')
print(f'Відносний lift: {(cr_trt-cr_ctrl)/cr_ctrl*100:+.1f}%')

# Z-тест
z_stat, p_val = proportions_ztest([x_trt, x_ctrl], [n_trt, n_ctrl], alternative='larger')

# Довірчий інтервал
diff = cr_trt - cr_ctrl
se = np.sqrt(cr_ctrl*(1-cr_ctrl)/n_ctrl + cr_trt*(1-cr_trt)/n_trt)
ci = (diff - 1.96*se, diff + 1.96*se)

print(f'\nz-stat = {z_stat:.4f}, p-value = {p_val:.4f}')
print(f'95% CI: [{ci[0]*100:.2f}%, {ci[1]*100:.2f}%]')
print()
print('Відхиляємо H0(різниця значуща)' if p_val < 0.05 else 'Не відхиляємо H0(різниця не значуща)')
```

```
Control CR: 3506/62568 = 5.60%
Test CR: 3731/62649 = 5.96%
Абс. різниця: +0.35 п.п.
Відносний lift: +6.3%
```

```
z-stat = 2.6681, p-value = 0.0038
95% CI: [0.09%, 0.61%]
```

Відхиляємо H0(різниця значуща)

```
# Графік конверсії та воронка
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

ax = axes[0]
```

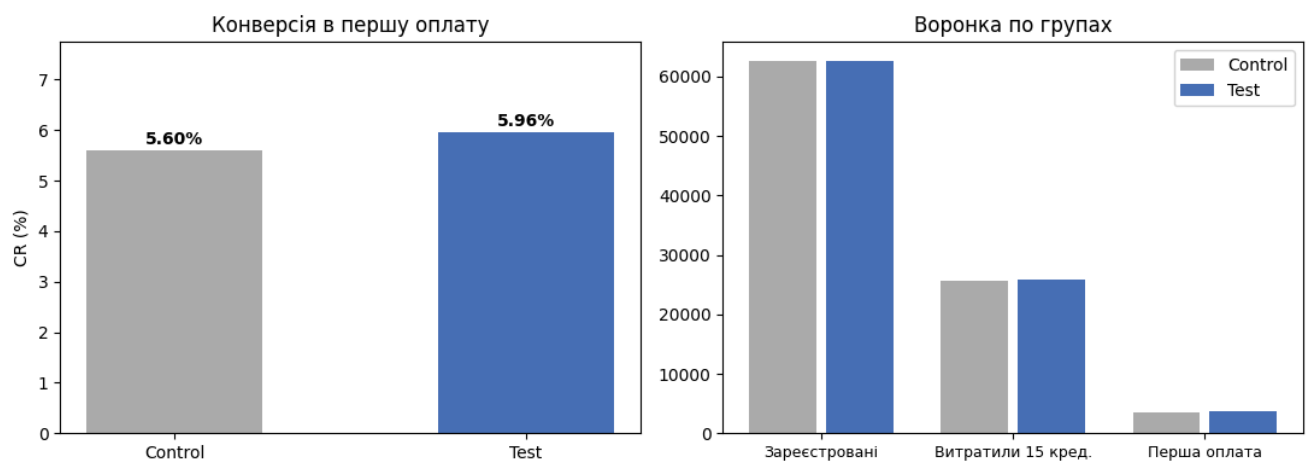
```

bars = ax.bar(['Control', 'Test'], [cr_ctrl*100, cr_trt*100],
              color=['#AAAAAA', '#4870B6'], width=0.5)
for bar, val in zip(bars, [cr_ctrl*100, cr_trt*100]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.05,
            f'{val:.2f}%', ha='center', va='bottom', fontweight='bold')
ax.set_title('Конверсія в першу оплату')
ax.set_ylabel('CR (%)')
ax.set_ylim(0, max(cr_ctrl, cr_trt)*100 * 1.3)

ax2 = axes[1]
steps = ['Зареєстровані', 'Витратили 15 кред.', 'Перша оплата']
ctrl_vals = [n_ctrl, ctrl['date_spent_15_credits'].notna().sum(), x_ctrl]
trt_vals = [n_trt, trt['date_spent_15_credits'].notna().sum(), x_trt]
x = np.arange(len(steps))
ax2.bar(x - 0.2, ctrl_vals, width=0.35, label='Control', color='#AAAAAA')
ax2.bar(x + 0.2, trt_vals, width=0.35, label='Test', color='#4870B6')
ax2.set_xticks(x)
ax2.set_xticklabels(steps, fontsize=9)
ax2.set_title('Воронка по групах')
ax2.legend()

plt.tight_layout()
plt.show()

```



Результати z-тесту:

	Control	Test
Розмір групи	62 568	62 649
Кількість оплат	3 506	3 731
CR	5.60%	5.96%
Абс. різниця		+0.35 п.п.
Відносний lift		+6.3%
p-value		0.0038
95% CI різниці		[+0.09%, +0.61%]

p-value = 0.0038 < 0.05 — відхиляємо H_0 . Різниця статистично значуща.

Довірчий інтервал **не включає 0** — це підтверджує, що результат не випадковий. Нагадування дало +6.3% відносний приріст конверсії.

Кількість зареєстрованих і тих, хто витратив 15 кредитів, практично однакова в обох групах — це очікувано, оскільки нагадування не впливає на активність до моменту, коли залишається ≤5 кредитів. Різниця проявляється саме на кроці першої оплати.

✓ Крок 3: Аналіз нагадування (date_reminder)

Перевіряємо якість імплементації тесту та дивимось, яка конверсія серед тих, хто фактично побачив нагадування.

```

# Аналіз нагадування
saw = trt[trt['date_reminder'].notna()]
not_saw = trt[trt['date_reminder'].isna()]

```

